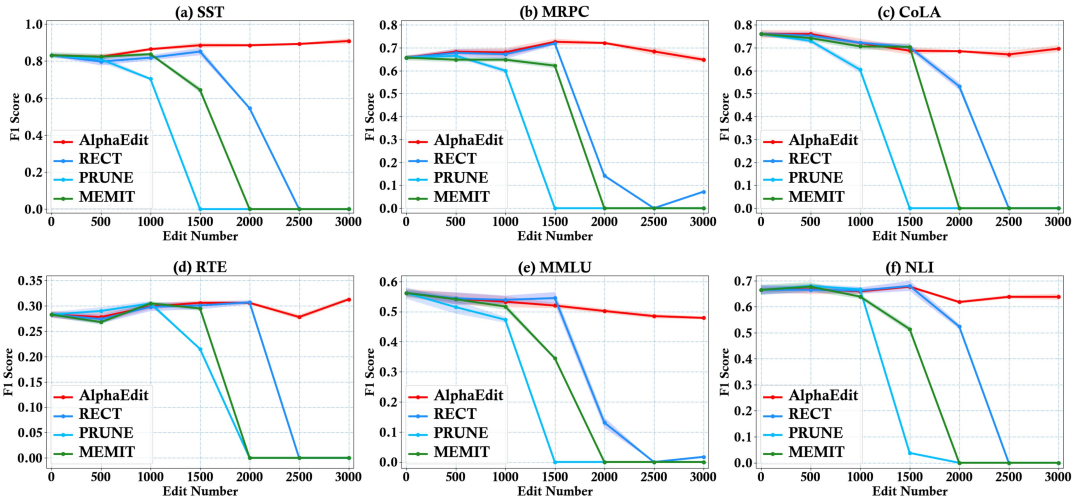

Figure 4: F1 scores of the post-edited LLaMA3 (8B) on six tasks (*i.e.*, SST, MRPC, CoLA, RTE, MMLU and NLI) used for general capability testing. Best viewed in color.

RECT (Gu et al., 2024). Furthermore, to further validate the generalizability of AlphaEdit, we include three memory-based editing methods (*i.e.*, SERAC (Mitchell et al., 2022b), GRACE (Hartvigsen et al., 2023) and MELO (Yu et al., 2024)) as baselines in Appendix C.4, along with two additional base LLMs: Gemma (Mesnard et al., 2024) and phi-1.5 (Li et al., 2023) in Appendix C.4.

Datasets & Evaluation Metrics. We evaluate AlphaEdit using two widely adopted benchmarks: the Counterfact dataset (Meng et al., 2022) and the ZsRE dataset (Levy et al., 2017). In line with prior works (Meng et al., 2022), we employ **Efficacy** (efficiency success), **Generalization** (paraphrase success), **Specificity** (neighborhood success), **Fluency** (generation entropy), and **Consistency** (reference score) as evaluation metrics. In addition, for comprehensive evaluation, Appendix C.7 presents experiments conducted on three additional datasets: LongformEvaluation (Rosati et al., 2024), MQUAKE (Zhong et al., 2023), and KnowEdit (Zhang et al., 2024d). We encourage interested readers to refer to Appendix C.7 for further details.

4.2 Performance on Knowledge Update and Preservation (RQ1)

To evaluate the performance of different editing methods in terms of knowledge update and retention, we conducted sequential editing on three base LLMs using AlphaEdit and the baseline methods. Table 1 presents the results under a commonly used configuration for the sequential editing task, where 2,000 samples are randomly drawn from the dataset for updates, with 100 samples per edit (*i.e.*, a batch size of 100). For additional experimental results, such as case studies of model outputs after editing, please refer to Appendix C. Based on Table 1, we can draw the following observations:

- **Obs 1: AlphaEdit achieves superior performance across nearly all metrics and base models.** Specifically, in terms of Efficacy and Generalization metrics, AlphaEdit provides an average improvement of 12.54% and 16.78%, respectively, over the best baseline. On LLaMA3, these performance boosts are even more remarkable (*i.e.,* $32.85\% \uparrow$ and $30.60\% \uparrow$). These gains arise from AlphaEdit's ability to mitigate the trade-off between updating and preserving knowledge.
- **Obs 2: AlphaEdit enhances text generation fluency and coherence.** In addition to editing capabilities, AlphaEdit also exhibits substantial improvements in Fluency and Coherence. For instance, on GPT2-XL, AlphaEdit achieves an 18.33% improvement over the strongest baseline, demonstrating that it can preserve both the knowledge and the ability to generate fluent text.

4.3 General Capability Tests (RQ2)

To further evaluate the intrinsic knowledge of post-edited LLMs, we perform General Capability Tests using six natural language tasks from the General Language Understanding Evaluation (GLUE) benchmark (Wang et al., 2019). Specifically, the evaluation tasks include:

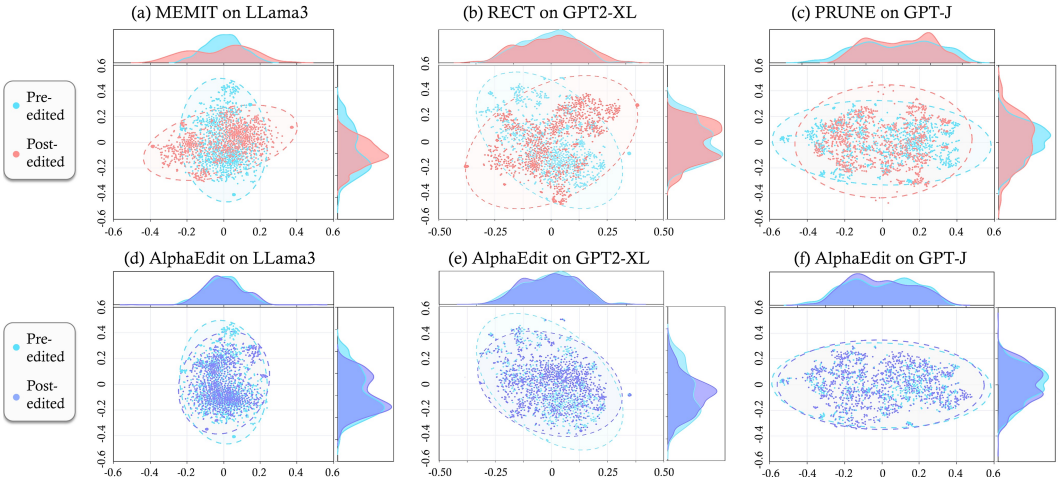

Figure 5: The distribution of hidden representations of pre-edited and post-edited LLMs after dimensionality reduction. The top and right curve graphs display the marginal distributions for two reduced dimensions, where AlphaEdit consistently exhibits minimal shift. Best viewed in color.

1. **SST (The Stanford Sentiment Treebank)** (Socher et al., 2013) is a single-sentence classification task involving sentences from movie reviews and their corresponding human-annotated sentiment labels. The task requires classifying the sentiment into two categories.
2. **MRPC (Microsoft Research Paraphrase Corpus)** (Dolan & Brockett, 2005) is a well-known benchmark for text matching and semantic similarity assessment. In the MRPC task, the objective is to determine whether a given pair of sentences is semantically equivalent.
3. **MMLU (Massive Multi-task Language Understanding)** (Hendrycks et al., 2021) is a comprehensive evaluation designed to measure the multi-task accuracy of text models. This assessment focuses on evaluating models under zero-shot and few-shot settings.
4. **RTE (Recognizing Textual Entailment)** (Bentivogli et al., 2009) involves natural language inference that determines if a premise sentence logically entails a hypothesis sentence.
5. **CoLA (Corpus of Linguistic Acceptability)** (Warstadt et al., 2019) is a single-sentence classification task, where sentences are annotated as either grammatically acceptable or unacceptable.
6. **NLI (Natural Language Inference)** (Williams et al., 2018) focuses on natural language understanding, requiring the model to infer the logical relationship between pairs of sentences.

Figure 4 illustrates the performance as the number of edited samples increases across six tasks. More results are provided in Appendix C.2. Based on Figure 4, we have the following observations:

- **Obs 3: AlphaEdit sustains the general capability of post-edited LLMs even after extensive editing.** Specifically, AlphaEdit maintains the original model performance across all metrics, even after editing 3,000 samples, demonstrating that the null-space projection not only safeguards the preserved knowledge but also protects the general capability learned from this knowledge's corpus.
- **Obs 4: LLMs edited with baseline methods experience significant degradation of general capability after editing 2,000 samples.** Specifically, in this case, all metrics are rapidly approaching zero, confirming our theoretical analysis that the common objective are inherently flawed and fail to balance knowledge update and preservation.

4.4 Hidden Representations Analysis (RQ3)

As discussed in previous sections, current editing methods often cause post-edited LLMs to overfit to the updated knowledge, leading to a shift in the distribution of hidden representations. Hence, here we aim to empirically verify that AlphaEdit can prevent overfitting and avoid this distributional shift. To validate it, we conducted the following steps: (1) We randomly select 1,000 factual prompts and extract the hidden representations within pre-edited LLMs. (2) Subsequently, we performed 2,000 sequential edits on the LLMs and recomputed these hidden representation. (3) Finally, we used t-SNE (Van der Maaten & Hinton, 2008) to visualize the hidden representation before and after editing. Figure 5 exhibits them and their marginal distribution curves. Furthermore, we quantify the

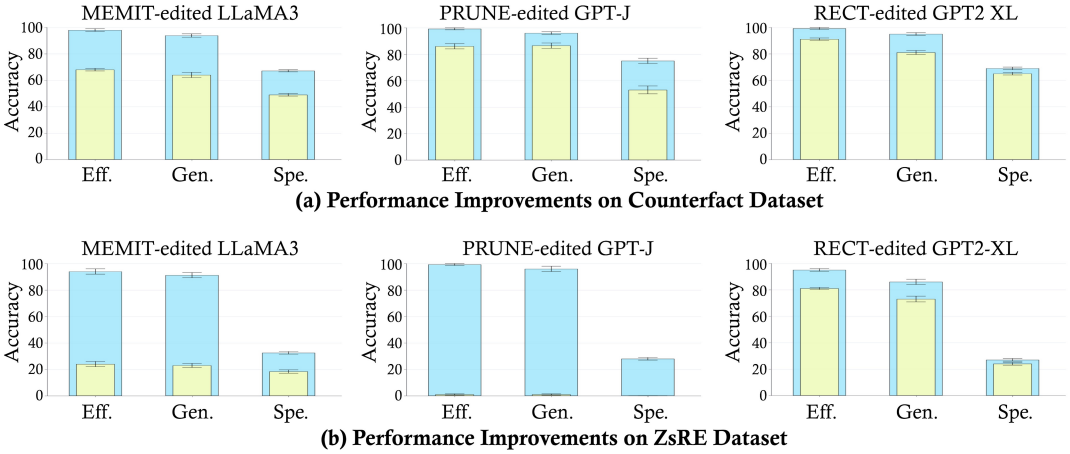

Figure 6: Performance improvements of baseline editing methods (*i.e.*, MEMIT, PRUNE and RECT) after **adding a single line of code from AlphaEdit** (*i.e.*, the code used for matrix projection). The yellow bars represent the original performance of each baseline, while the blue bars represent the performance after the addition. Best viewed in color.

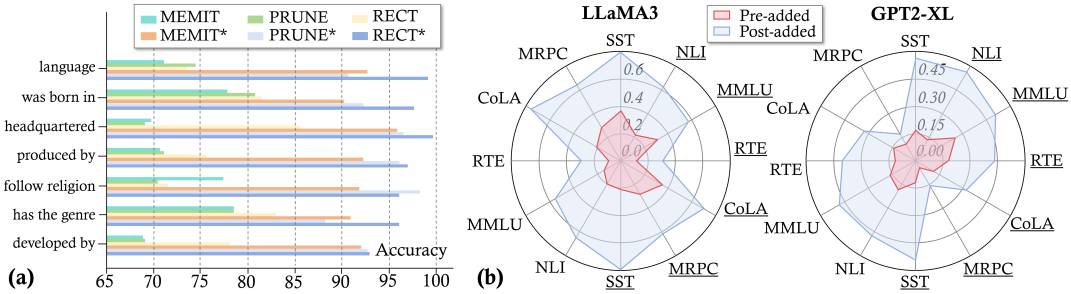

Figure 7: Performance comparison before and after adding the projection code in AlphaEdit to the baselines. (a) Performance of editing methods on samples involving specific semantics, where asterisk denotes the versions added the projection; the vertical and horizontal axis represent the categories of knowledge and the accuracy of LLM responses involving this knowledge, respectively. (b) Comparison of general capabilities for PRUNE and RECT before and after adding the projection, where the underlined method represents the results of RECT. Best viewed in color.

deviation of scatter point distributions and the differences of marginal distribution curves in Figure 5, and the results are provided in Appendix C.3. According to Figure 5 we can find that:

- **Obs 5: AlphaEdit maintains consistency in hidden representations after editing.** Specifically, the hidden representations within LLMs edited using AlphaEdit remain consistent with the original distribution across all three base models. This stability indicates that AlphaEdit effectively mitigates overfitting, in turn explaining its superior knowledge preservation and generalization capabilities.
- **Obs 6: There is a significant shift in the distribution of hidden representations within baseline-edited LLMs.** In some cases (*e.g.,* RECT-edited LLaMA3), the trend of the distribution before and after editing is even completely reversed. This discrepancy becomes more pronounced as sequential editing progresses, further underscoring the importance of projection-based optimization.

4.5 Performance Improvements of Baseline Methods (RQ4)

We conclude by evaluating whether integrating AlphaEdit's projection strategy can comprehensively enhance current editing methods. To achieve this, we add one line of code from AlphaEdit (the code for projection) to baselines and measure their performance before and after the addition. Following previous works (Meng et al., 2023), we analyze (1) the average performance across all editing